

3B : RISC-V Microarchitecture

Rajesh Panicker
ECE, NUS

CG3207

Note:

- V2:
 - Additional info / clarifications / enhancements in purple colour
 - Slides 3, 4 added for additional info / clarifications. Slide 4 in V1 is split into two slides – 6 and 7, with info added in purple colour



Basic RISC-V Microarchitecture

- We are building a processor that supports only a subset of RV32I
- For a start, we DO NOT support
 - Conditional branches other than beq
 - lui and auipc
 - jalr
 - link functionality (saving return address) of jal
 - byte/half-word load and store, i.e., we support only lw and sw
- We will add some of the above functionalities later, gradually



Microarchitecture - Datapath

- Datapath – the path through which data ‘flows’. Includes
 - Storage elements - memories and registers
 - Architectural state elements – manipulated by the programmer: instruction memory, data memory, register file (x0-x31), PC, other control registers
 - Microarchitectural state elements – not accessible to the programmers: pipeline registers, cache tags, and branch predictor state
 - Steering logic – to channel data appropriately
 - Multiplexers and internal buses
 - Functional units – operate on data
 - ALUs, adders (often stateless)
 - Interface resources – interconnect various blocks
 - External buses and ports

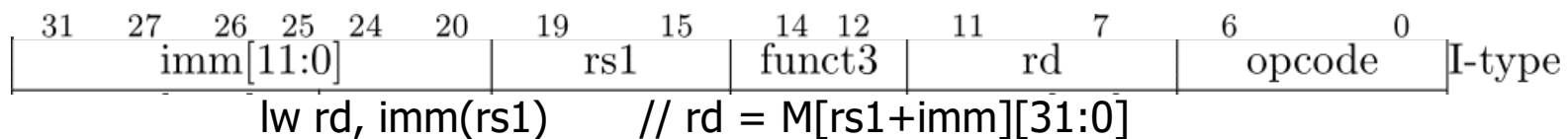
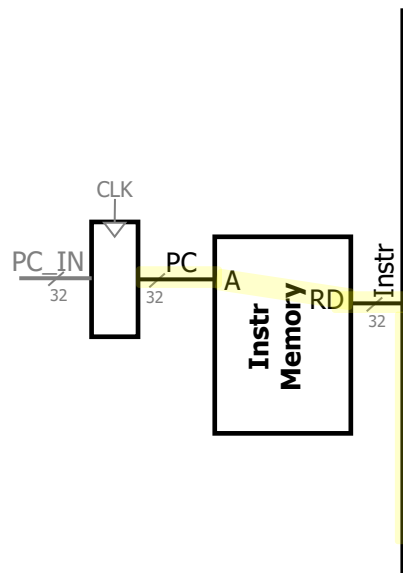


Microarchitecture – Control

- Control Unit controls the flow/processing/storage of data in the datapath via
 - Mux selects
 - Register write enables
 - Functional unit activation/enable signals and operation selection signals (e.g., to specify whether ALU should do addition / subtraction etc.)
- Usually a state machine of some sort
 - Hardwired – implemented as a usual state machine
 - Faster, inflexible
 - Microprogrammed – implemented as a counter + ROM
 - Slower, bug fixes possible after production if the ROM (control store) is writable
 - As simple as a decoder for a single-cycle microarchitecture
 - Fetch, decode and execute an instruction in a single clock cycle

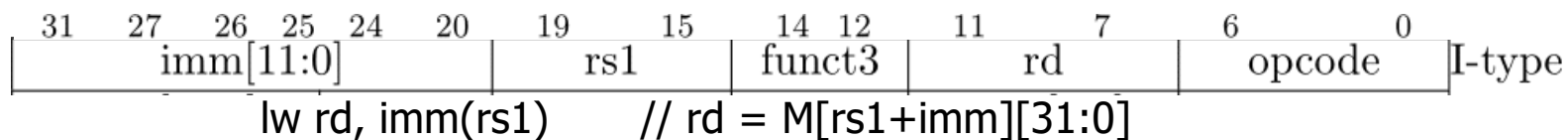
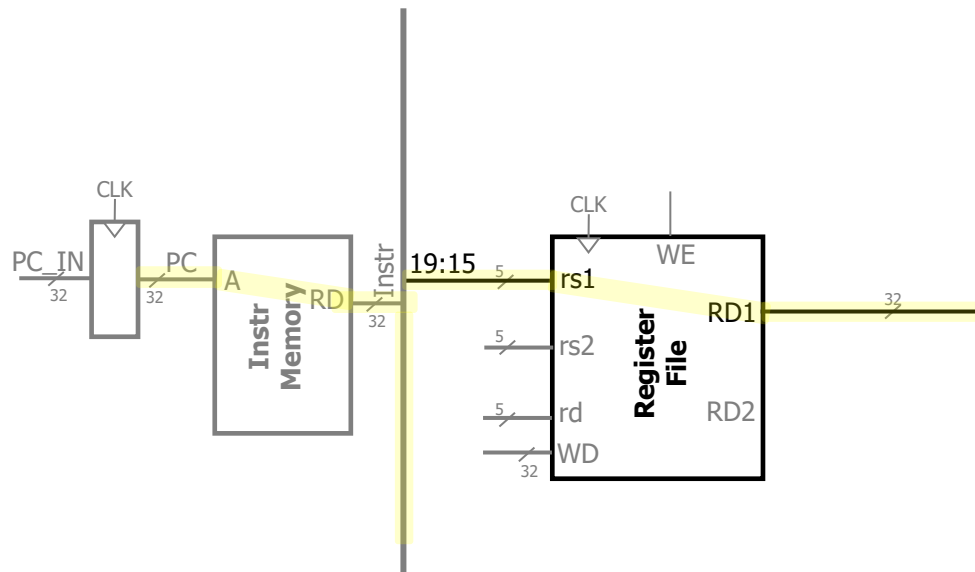
lw – Read Instruction Memory

- We start a single-cycle microarchitecture with lw instruction
- # 1: Read instruction memory
 - Assume instruction memory can be read combinatorially – no clock required



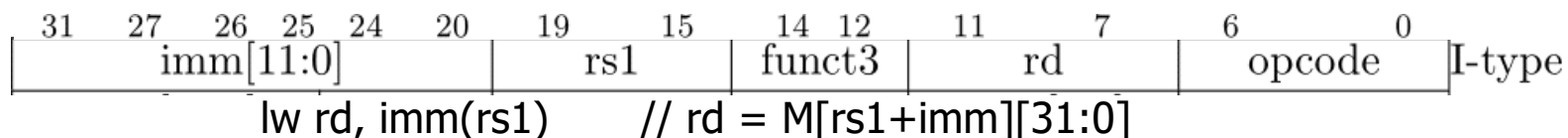
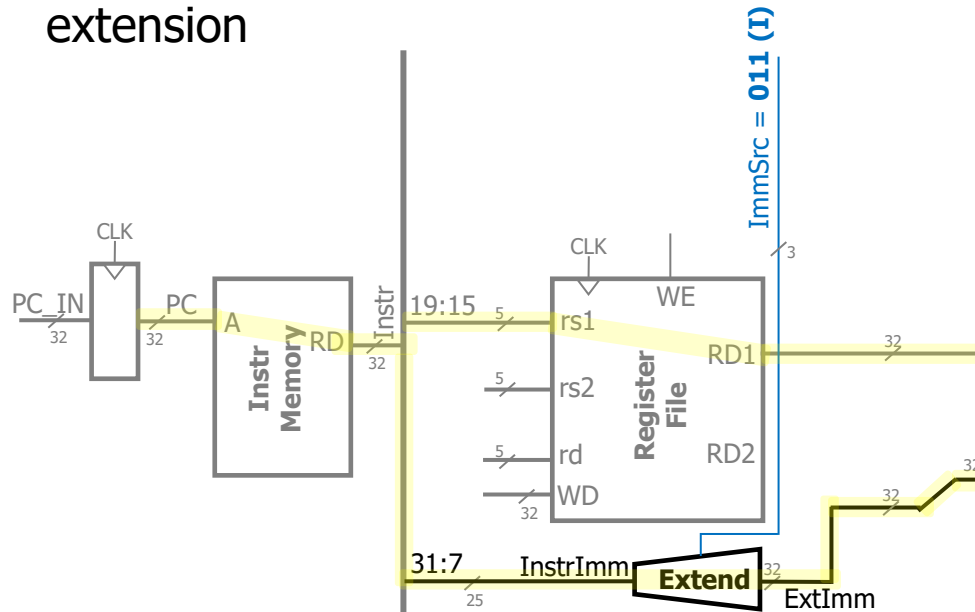
lw – Read Base Register

- #2: Read the register file (rs1) for the base address (first source operand, RD1) – no clock needed for reading



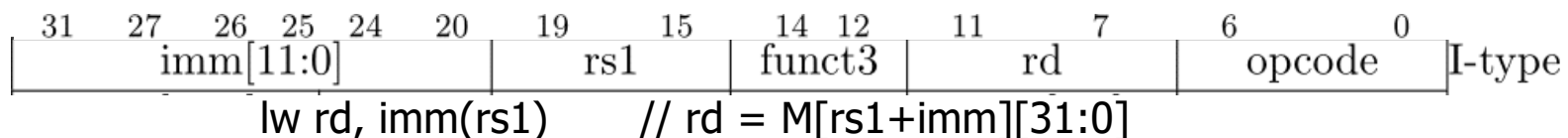
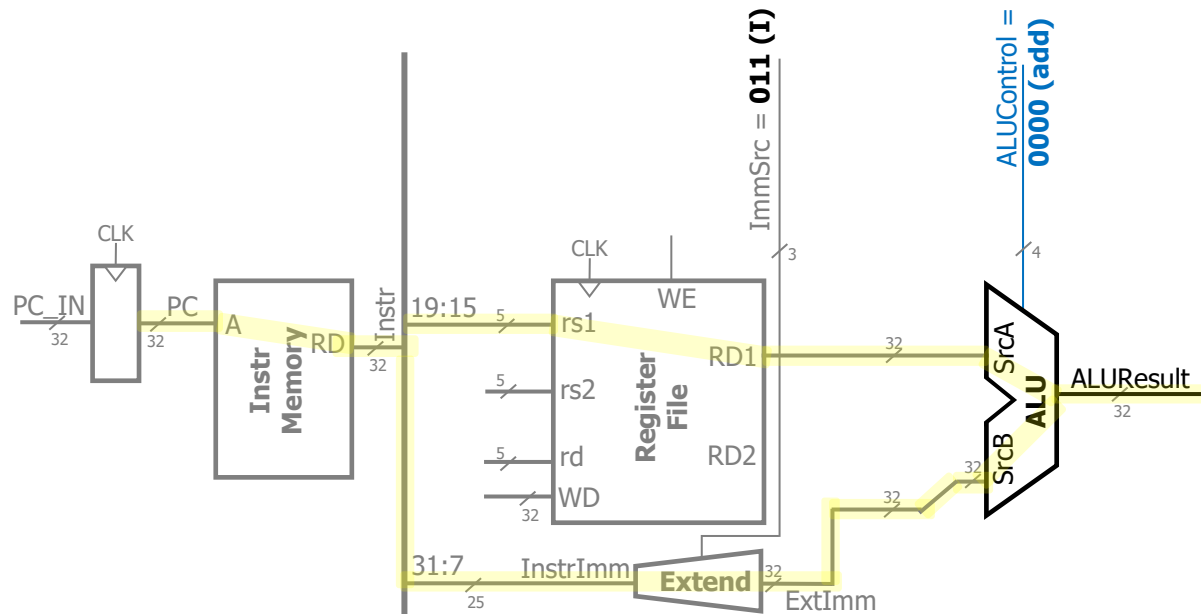
lw – Extend imm

- #3: Sign-extend the immediate to form the offset (second operand, ExtImm)
 - There are 5 different formats for immediates (I, S, B, U, J). The Extend unit needs a 3-bit ImmSrc control signal to specify the type of extension



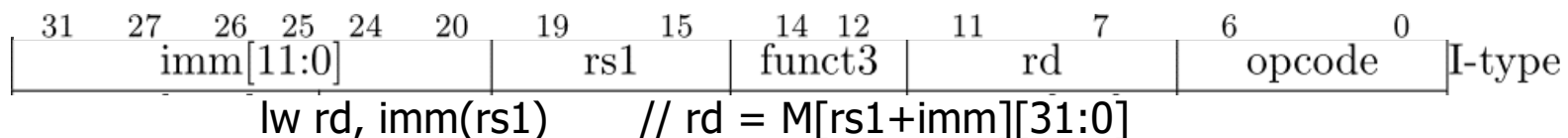
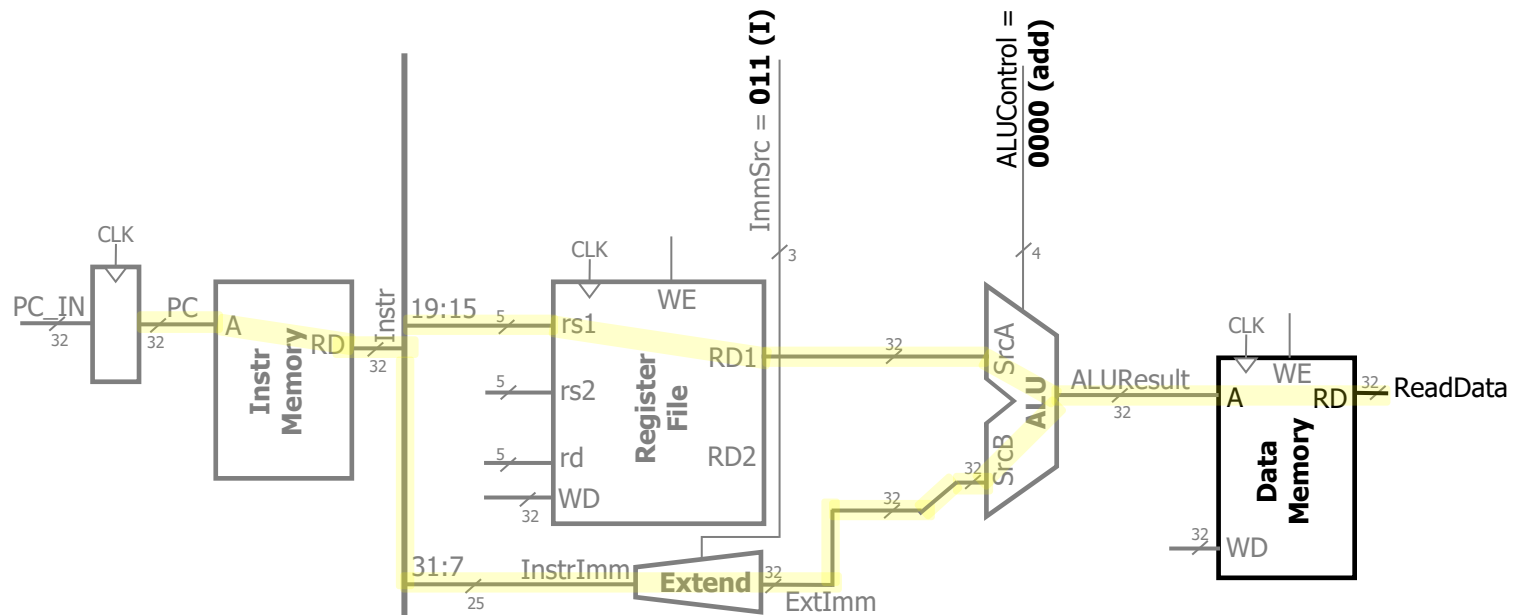
lw – Compute Address

- #4: Compute the effective data memory address using ALU
 - ALU does addition



lw – Read Data Memory

- #5: Read the data memory using the computed address
 - No clock needed

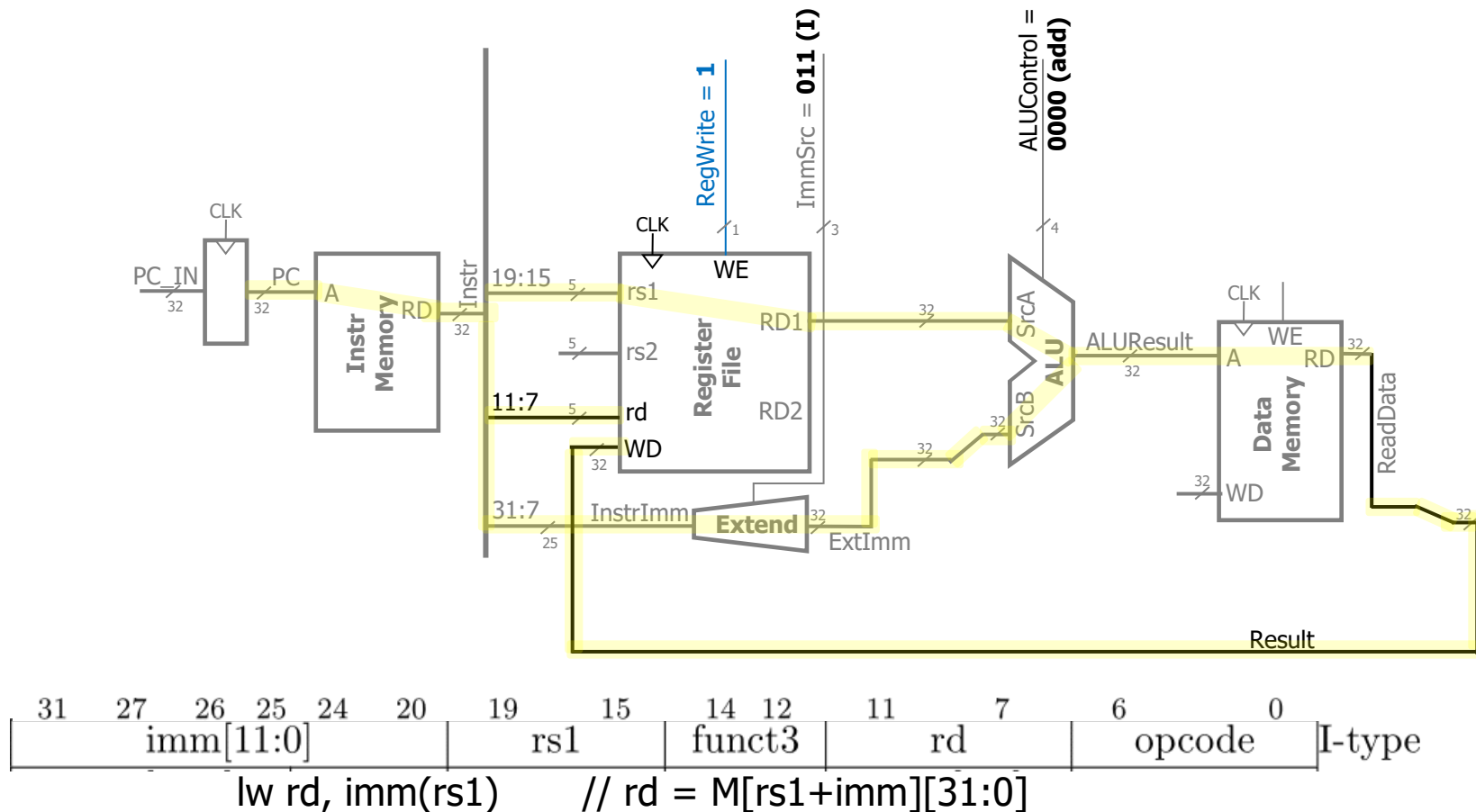


lw – Write Destination Register

Writing a register in the register file involves asserting the write enable of one of the 32 registers: WE AND 5 to 32 decoder.
WD is given to all 32 registers

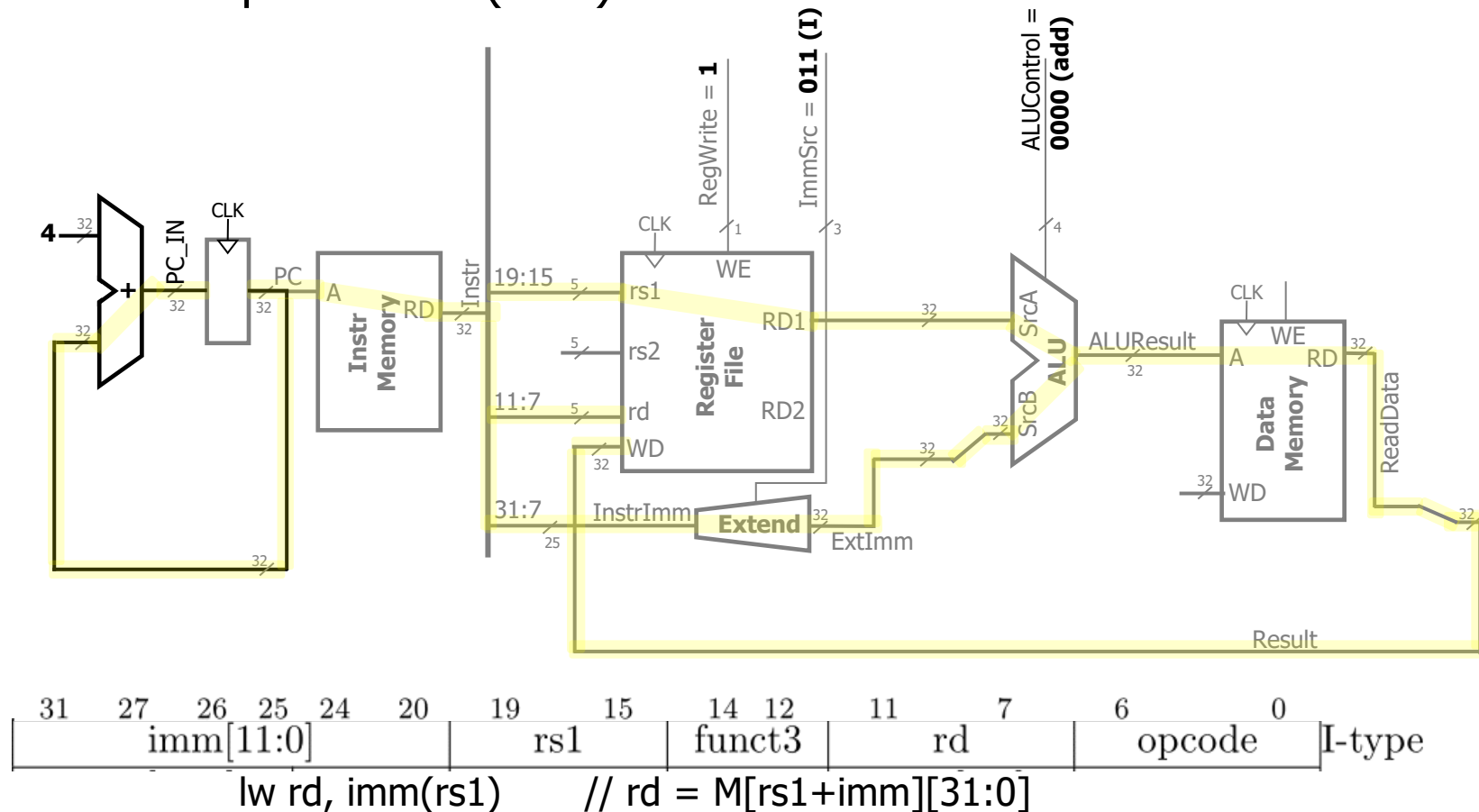
■ #6: Write the result into the destination register

- *what* : Result *where to* : rd *whether* : RegWrite
- *when* : **active** clock edge – writing needs clock!

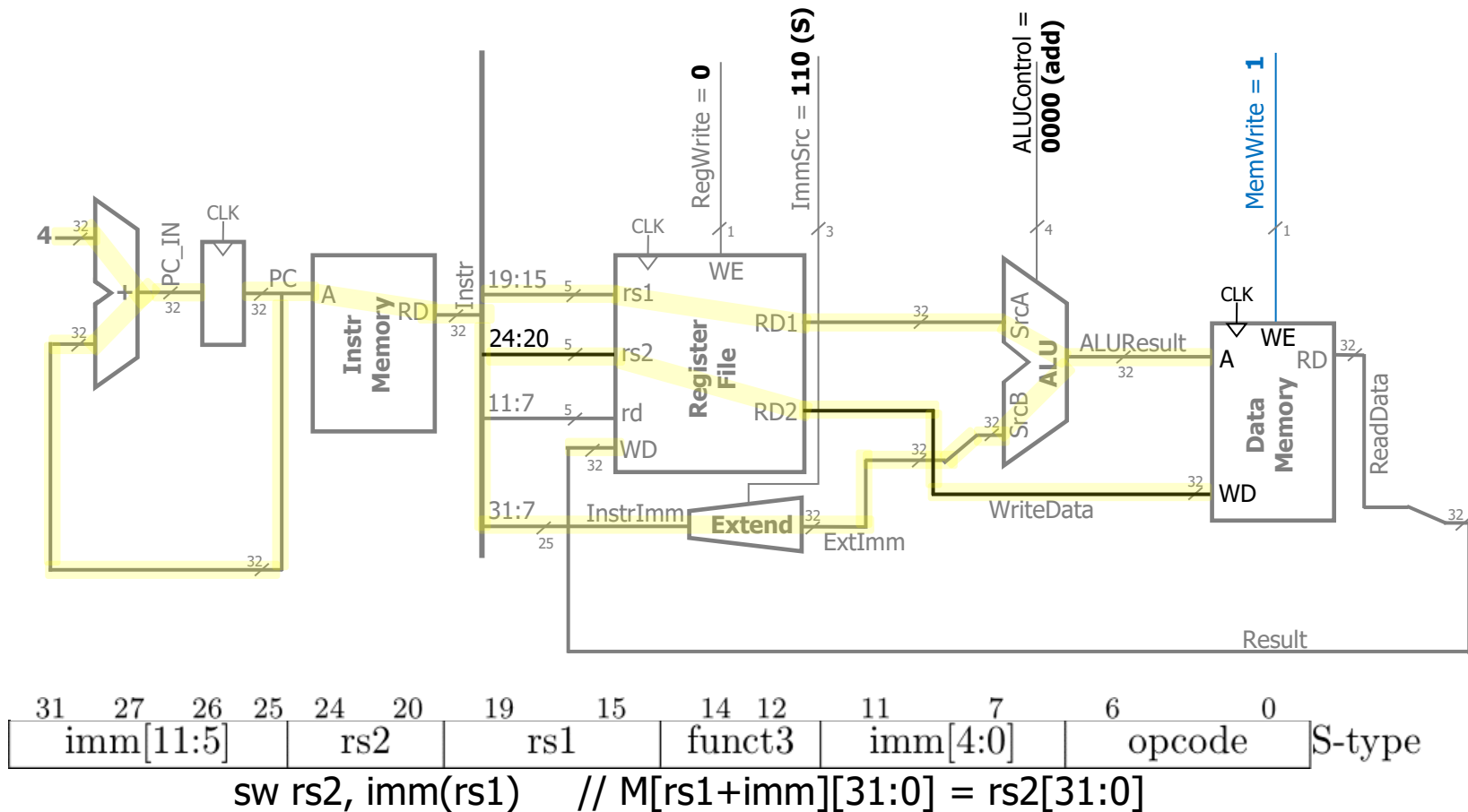


lw – PC Increment

- #7: Increment PC by 4 – all our instructions (RV32I) are 32 bits (4 bytes) long
 - PC written at the same clock edge as rd write
- lw complete! New (next) instruction fetch starts

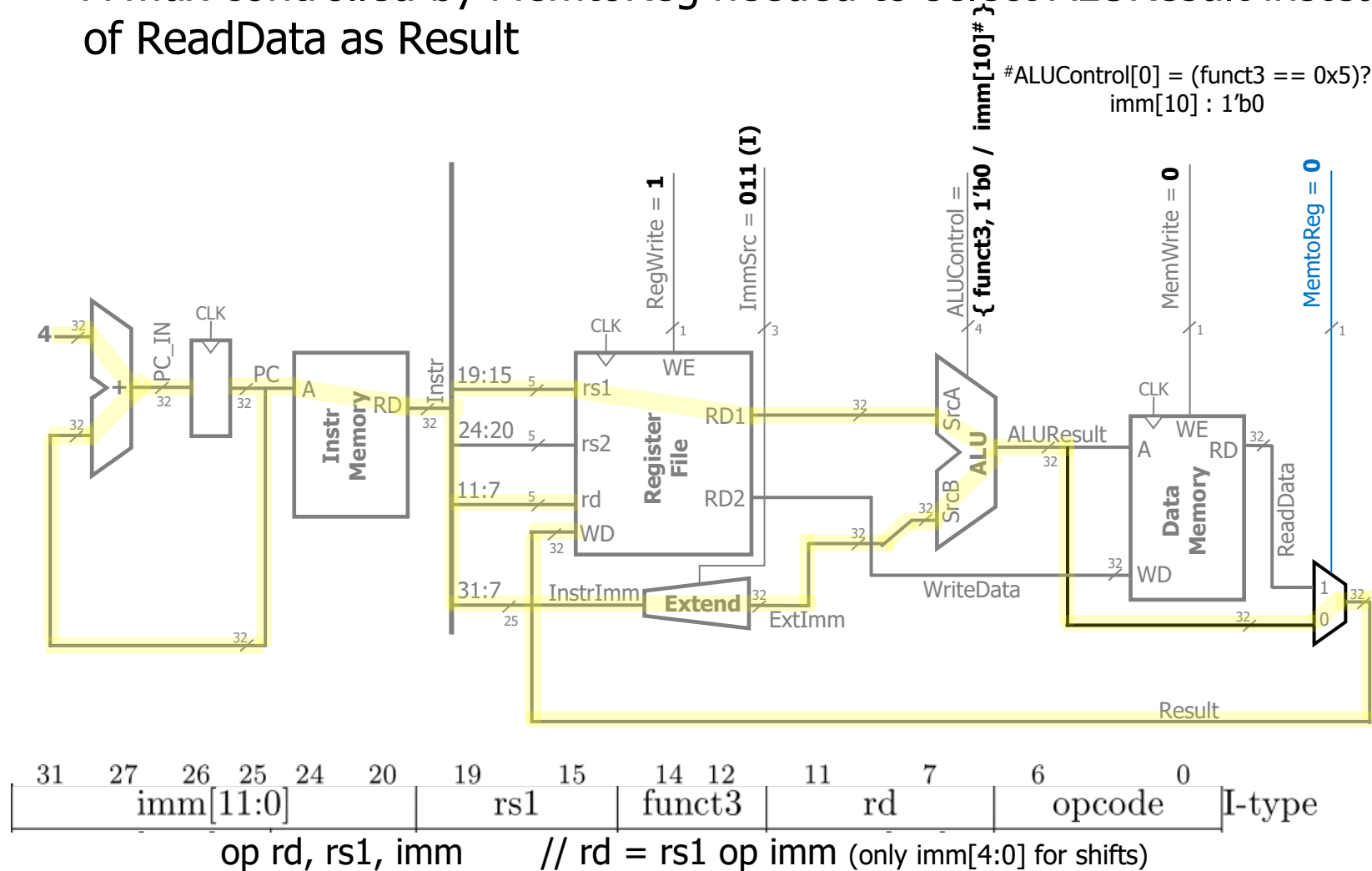


- Write the content of rs2 into memory
 - what* : rs2 content (RD2) *where to* : $M[ALUResult]$ *whether* : MemWrite
 - when* : same clock edge as PC write



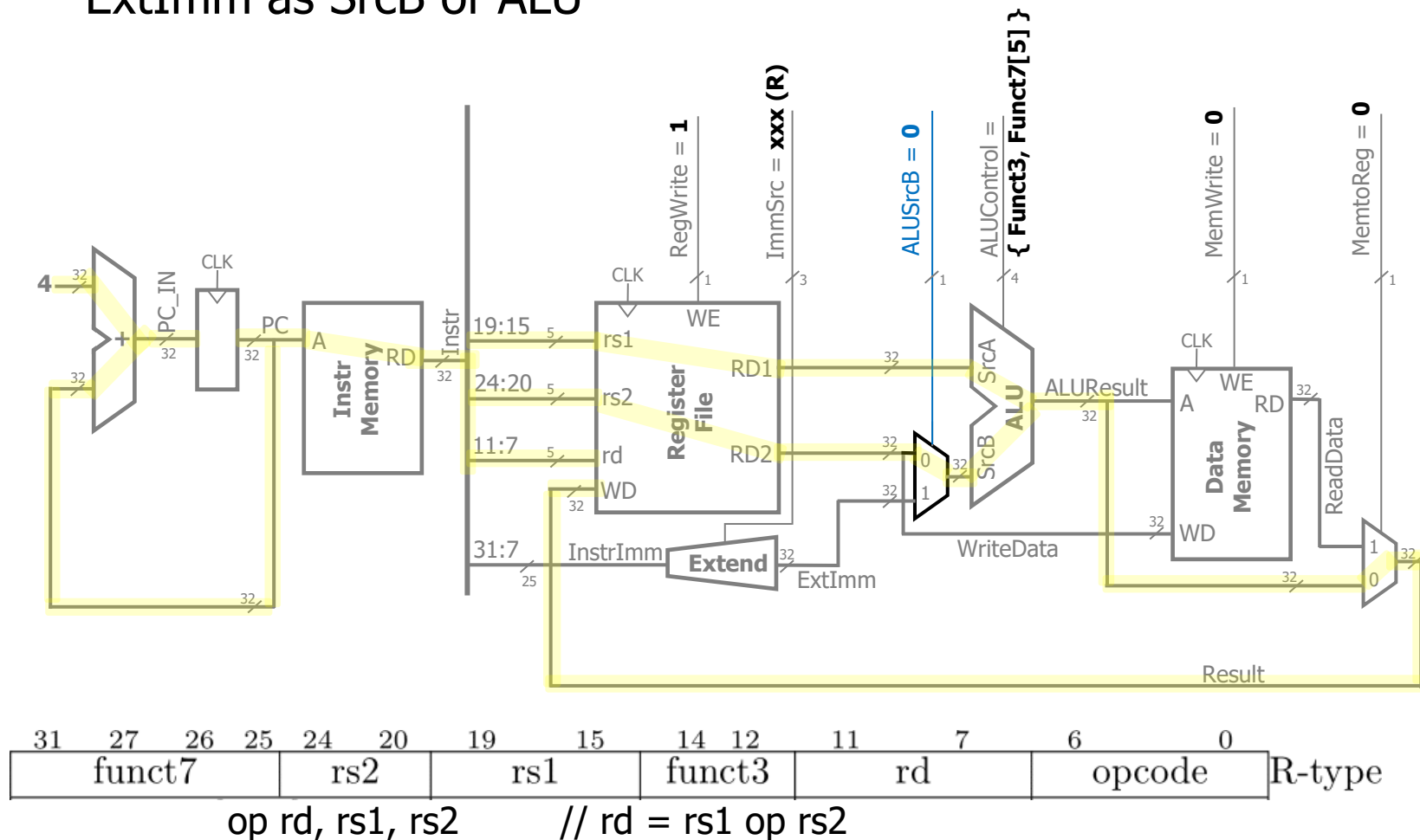
DP Immediate

- ALU does op on rs1 content (RD1) and ExtImm
- A mux controlled by MemtoReg needed to select ALUResult instead of ReadData as Result

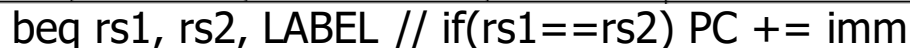
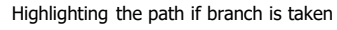


DP Register

- ALU does op on rs1 content (RD1) and rs2 content (RD2)
- A mux controlled by ALUSrcB needed to select RD2 instead of ExtImm as SrcB of ALU

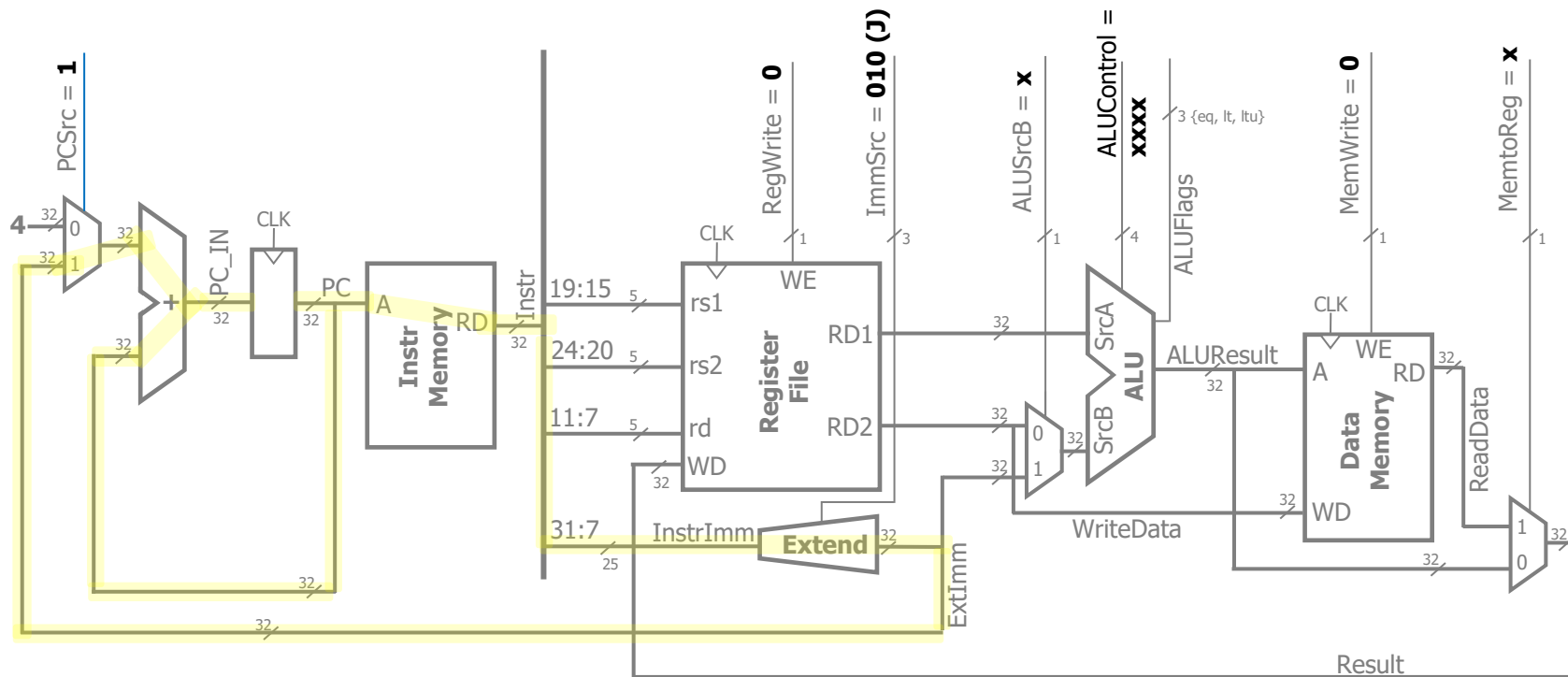


- 1 if the branch
is taken \



jal

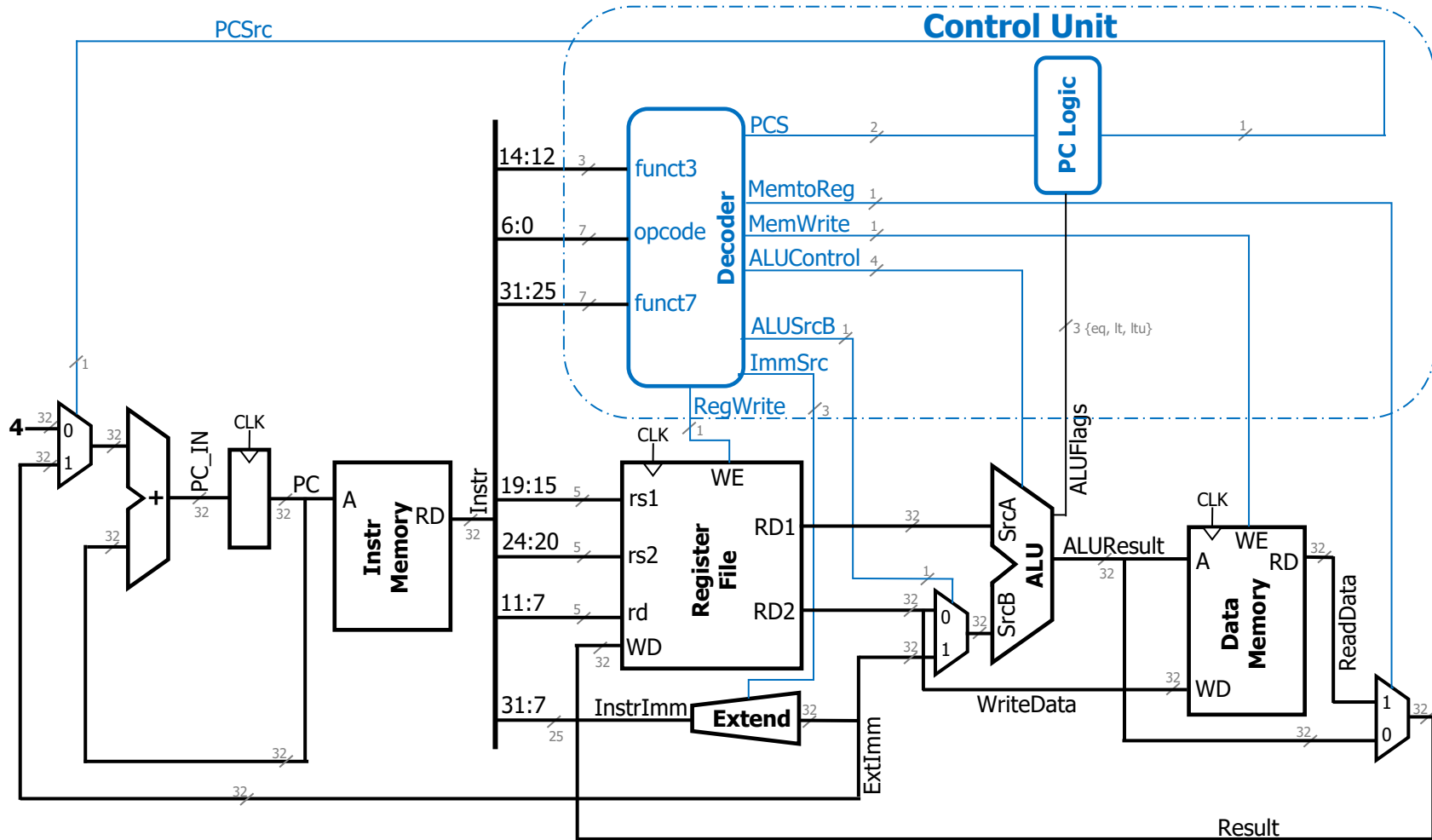
- No additional datapath hardware needed
 - The Extend unit extends the immediate differently though



31 27 26 25 24 20 19 15 14 12 11 7 6 0
 imm[20|10:1|11|19:12] rd opcode J-type

`jal rd, LABEL // PC += imm; rd = PC+4 (Our design doesn't support link for now)`

Single-Cycle Processor with Control



Single-Cycle Processor Control Unit

Opcode (7-bit hex)	Instruction	PCS	MemtoReg	RegWrite	MemWrite	ALUSrcB	ImmSrc	ALUControl
33	DP Reg	00	0	1	0	0	xxx (R)	{ funct3, funct7[5] }
13	DP Imm	00	0	1	0	1	011 (I)	{ funct3, 1'b0 / imm[10] [#] }
03	load	00	1	1	0	1	011 (I)	0000 (add)
23	store	00	x	0	1	1	110 (S)	0000 (add)
63	branch	01	x	0	0	0	111 (B)	0001 (sub)
6F	jal	10	x	0	0	x	010 (J)	xxxx

Decoder

#ALUControl[0] = (funct3 == 0x5)? imm[10] : 1'b0
imm[10] is the same bit as funct7[5], which is Instr[30]

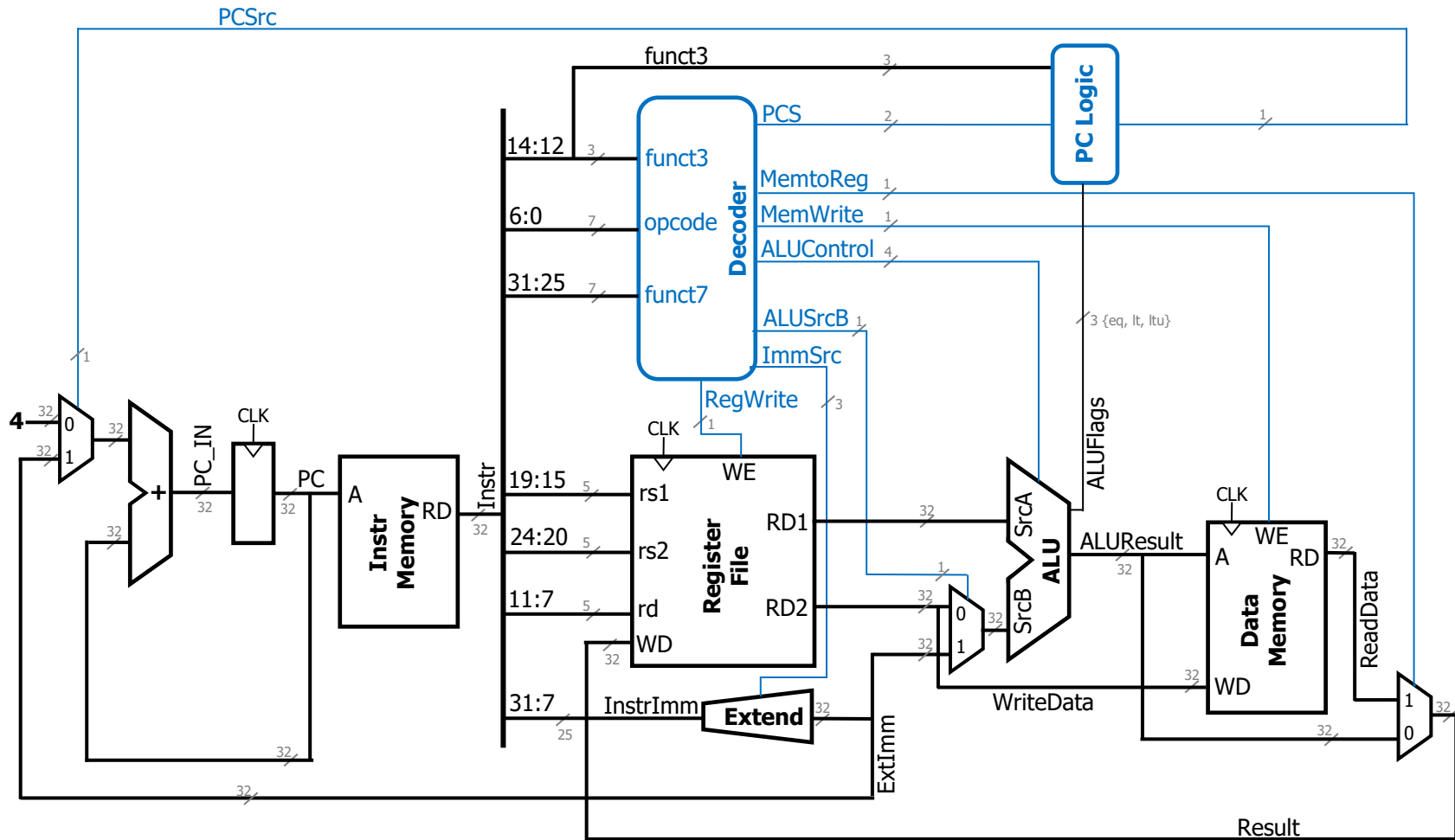
PCS	Instruction	PCSrc
00	Non control	0
01	beq	ALUFlags[2]
10	jal	1

PC Logic

ALUFlags[2:0] is {eq, lt, ltu}

Here, we do not differentiate between various branch instructions. All are treated as beq

PC Logic With All Conditions – Datapath



PC Logic With All Conditions – Control

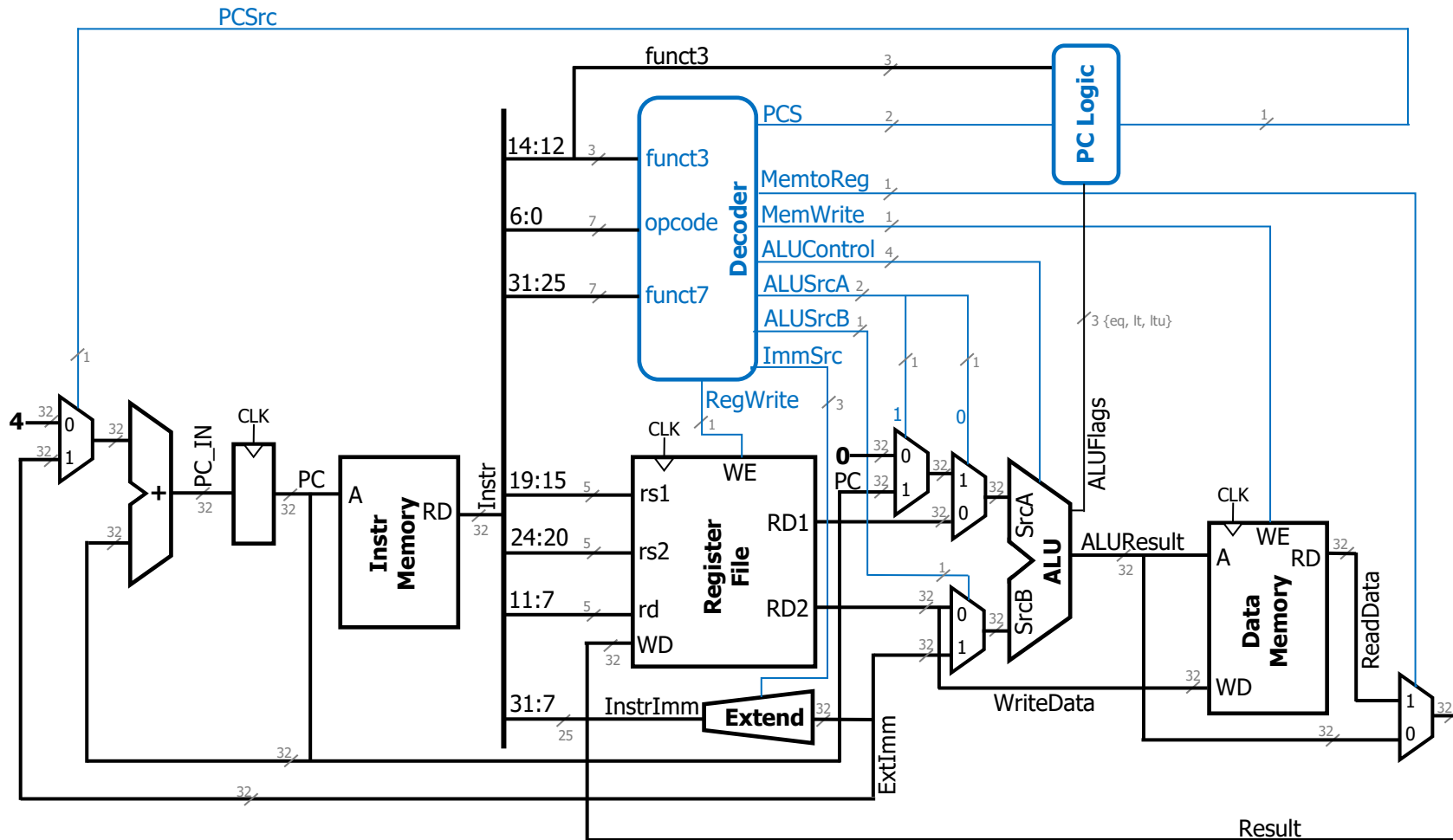
Instruction	PCS	Funct3	PCSrc
Non control	00	x	0
beq	01	000	ALUFlags[2]
bne	01	001	ALUFlags[2]'
blt	01	100	ALUFlags[1]
bge	01	101	ALUFlags[1]'
bltu	01	110	ALUFlags[0]
bgeu	01	111	ALUFlags[0]'
jal	10	x	1

PC logic that differentiates between various branch instructions

ALUFlags[2:0] is {eq, lt, ltu}

The ALU/hardware does not know whether the numbers it is dealing with are signed or unsigned. The programmer should know the signedness and use the correct instruction (blt for signed vs bltu for unsigned, etc).

Support for lui and auipc – Datapath

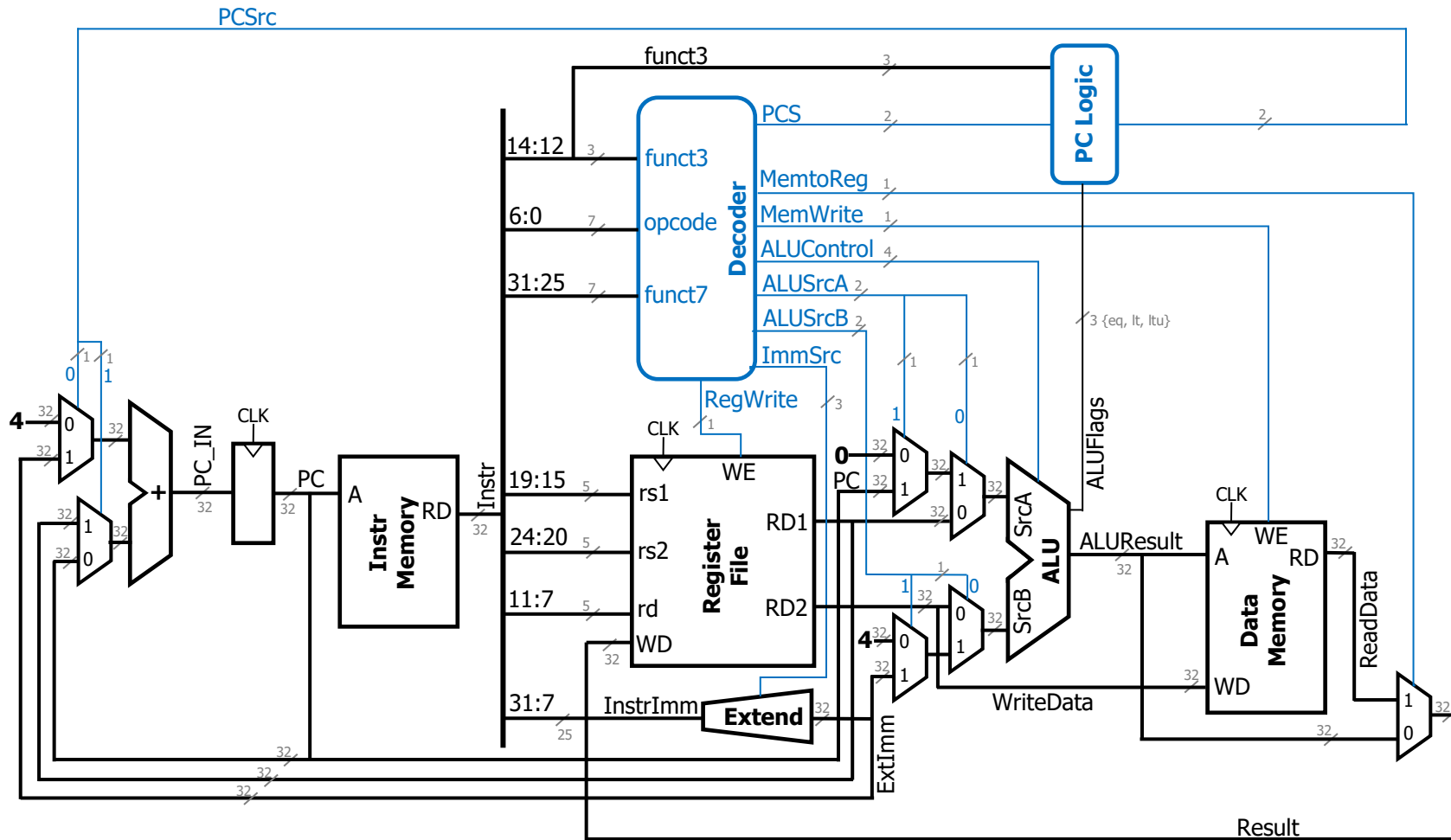


Support for lui and auipc – Control

Opcode (7-bit hex)	Instruction	PCS	MemtoReg	RegWrite	MemWrite	ALUSrcA	ALUSrcB	ImmSrc	ALUControl
33	DP Reg	00	0	1	0	x0	0	xxx (R)	{ funct3, funct7[5] }
13	DP Imm	00	0	1	0	x0	1	011 (I)	{ funct3, 1'b0 / imm[10]# }
03	load	00	1	1	0	x0	1	011 (I)	0000 (add)
23	store	00	x	0	1	x0	1	110 (S)	0000 (add)
63	branch	01	x	0	0	x0	0	111 (B)	0001 (sub)
6F	jal	10	x	0	0	xx	x	010 (J)	xxxx
17	auipc	00	0	1	0	11	1	000 (U)	0000 (add)
37	lui	00	0	1	0	01	1	000 (U)	0000 (add)

#ALUControl[0] = (funct3 == 0x5)? imm[10] : 1'b0

Support for link and jalr – Datapath



Note: Advantageous to have just 1 level of multiplexing for RD1 and RD2 as they will likely have more delay compared to PC and ExtImm respectively.

Support for link and jalr – Control

Opcode (7-bit hex)	Instruction	PCS	MemtoReg	RegWrite	MemWrite	ALUSrcA	ALUSrcB	ImmSrc	ALUControl
33	DP Reg	00	0	1	0	x0	x0	xxx (R)	{ funct3, funct7[5] }
13	DP Imm	00	0	1	0	x0	11	011 (I)	{ funct3, 1'b0 / imm[10]# }
03	load	00	1	1	0	x0	11	011 (I)	0000 (add)
23	store	00	x	0	1	x0	11	110 (S)	0000 (add)
63	branch	01	x	0	0	x0	x0	111 (B)	0001 (sub)
6F	jal	10	0	1	0	11	01	010 (J)	0000 (add)
17	auipc	00	0	1	0	11	11	000 (U)	0000 (add)
37	lui	00	0	1	0	01	11	000 (U)	0000 (add)
67	jalr	11	0	1	0	11	01	011 (I)	0000 (add)

#ALUControl[0] = (funct3 == 0x5)? imm[10] : 1'b0

Support for link and jalr – PC Logic

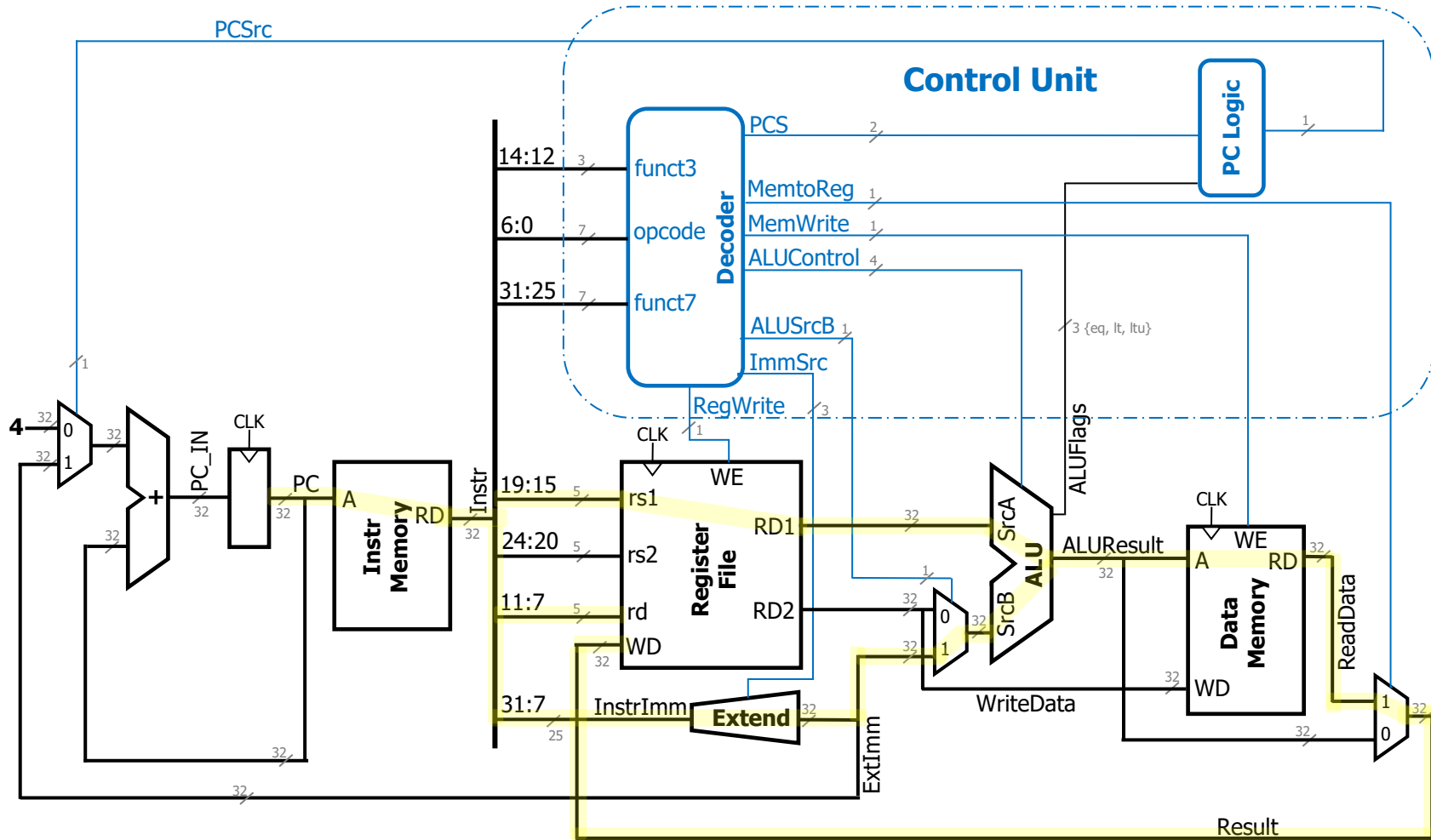
PCS	Instruction	Funct3	PCSrc[1]	PCSrc[0]
00	Non control	x	0	0
01	beq	000	0	ALUFlags[2]
01	bne	001	0	ALUFlags[2]'
01	blt	100	0	ALUFlags[1]
01	bge	101	0	ALUFlags[1]'
01	bltu	110	0	ALUFlags[0]
01	bgeu	111	0	ALUFlags[0]'
10	jal	x	0	1
11	jalr	x	1	1

ALUFlags[2:0] is {eq, lt, ltu}

PCSrc[1] selects base for PC_IN : RD1 or PC

PCSrc[0] controls offset for PC_IN : ExtImm or 4

Single-Cycle Processor Performance



- T_C (and hence, performance) limited by critical path (lw)

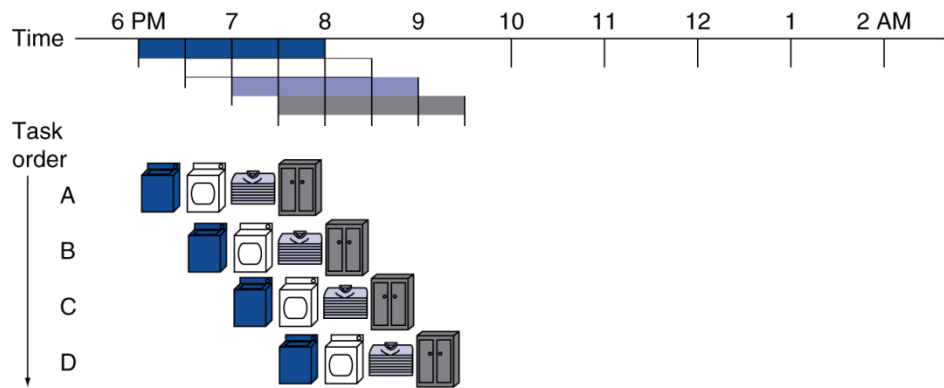
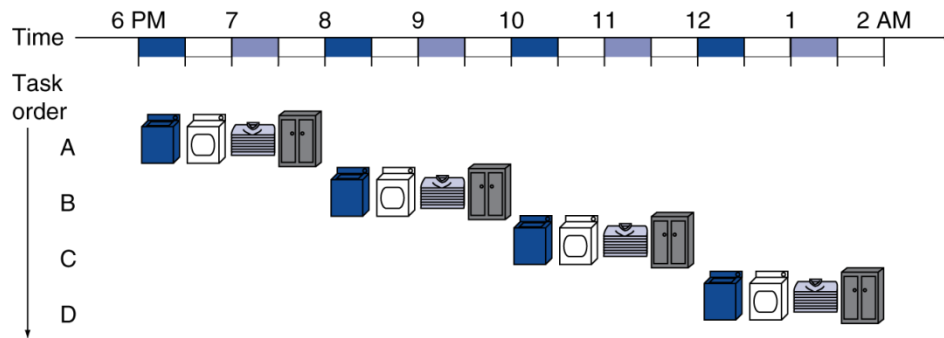


Single Cycle Design Summary

- **Single-cycle** - fetch, decode and execute each instruction in **one** clock cycle
 - (+) simple
 - (–) no datapath resource can be used more than once per instruction, so some must be duplicated
 - separate memories for instruction and data
 - 2 adders – one in ALU, one for PC_IN computation
 - (–) cycle time limited by longest instruction (LDR)
- How Can We Make It Faster?
 - Pipelining
 - Multiple-issue
 - Multithreading
 - Multiprocessor
 - stay tuned!

Pipelining Analogy. Coming Soon!

- Pipelined laundry: overlapping execution
 - Overlapped execution improves performance



- Four loads:
 - Speedup
 $= 8 / 3.5 = 2.3$
- Non-stop:
 - Speedup
 $= 2n / (0.5n + 1.5)$
 $\approx 4 = \# \text{ of stages}$